

# Bank Database Management System

**Instructor**

Dr. Priyamvada

**Submitted By**

Veer Doria 240041039

Naveel Singh 240041028

Sahil 240041032

Katravath Madhavi 240041021

# Problem Definition & Scope

- **System Description:** The "Children's Bank of India" is a Database Management System designed to automate core banking operations. It bridges the gap between customer banking needs and administrative oversight through a secure, dual-portal web application.
- **Scope of the Application:** The system facilitates end-to-end digital banking, including user onboarding, account lifecycle management (opening, freezing, closing), and financial ledger tracking.

# Core Entities & Assumptions

- **Major Entities:**

- **Users:** Stores demographic and authentication data (Customers and Admins).
- **Accounts:** Represents distinct financial holdings (Savings, Current) linked to users.
- **Transactions:** The immutable ledger tracking money movement.
- **Loans:** Manages debt profiles, interest rates, and EMI schedules.

- **System Assumptions:**

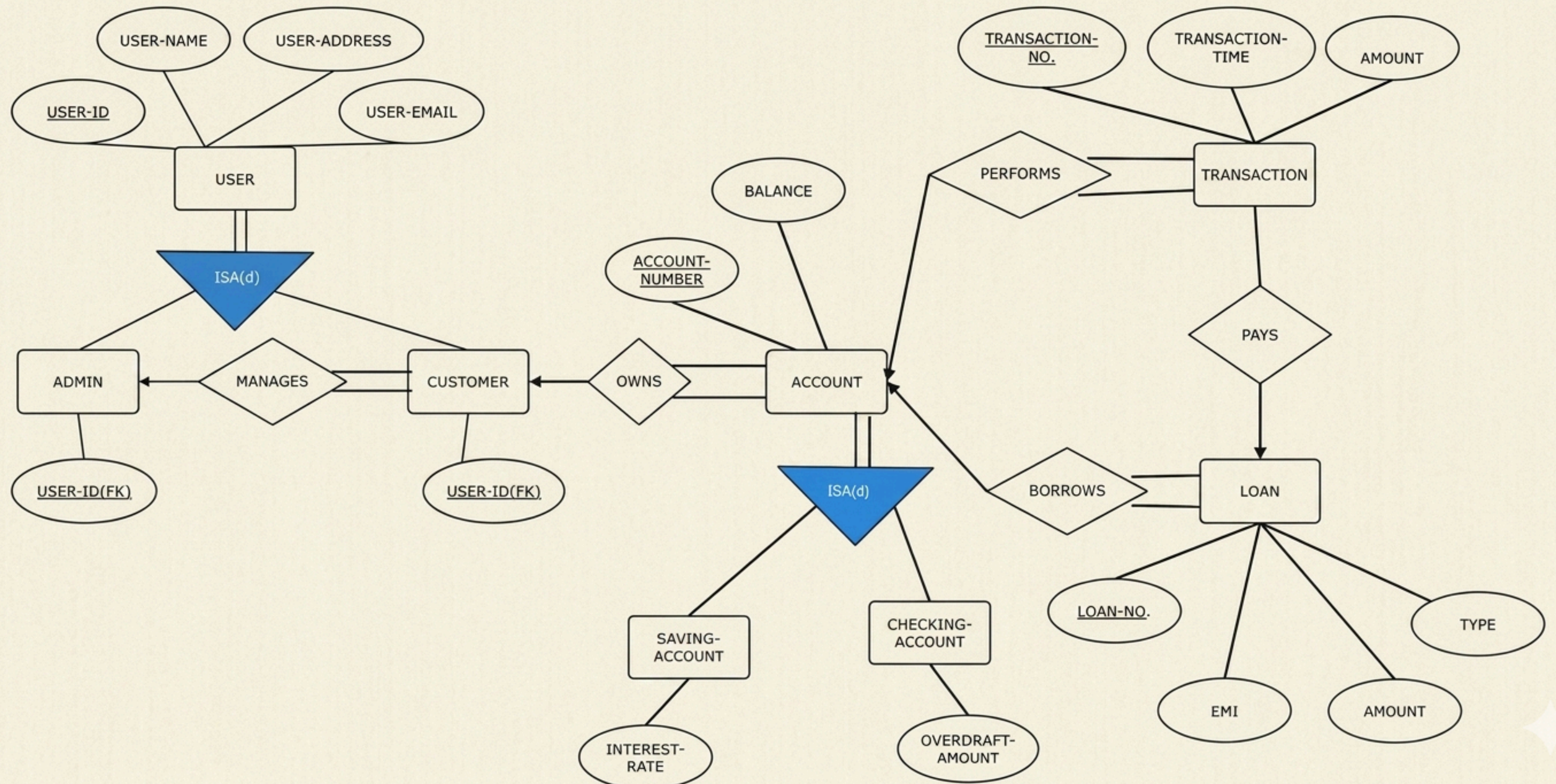
- The system operates using a single unified currency (INR / ₹).
- Customers must receive explicit Admin approval before an loan issues and can execute transfers.

# Key Functionalities

- **Expected System Functionalities:**
  - **Secure Authentication:** JWT-based login with strict Role-Based Access Control (Admin vs. Customer).
  - **Account Management:** Secure Account by several status (Running or Closed).
  - **Transaction Processing:** Atomic database transactions ensuring money is safely transferred between sender and receiver accounts without data corruption.
  - **Loan Operations Engine:** Automated calculation of EMIs, fee deductions, and loan lifecycle tracking (Pending -> Active -> Closed).
  - **Administrative Oversight:** A real-time statistical dashboard, dynamic user directory search, and an integrated SQL developer console.

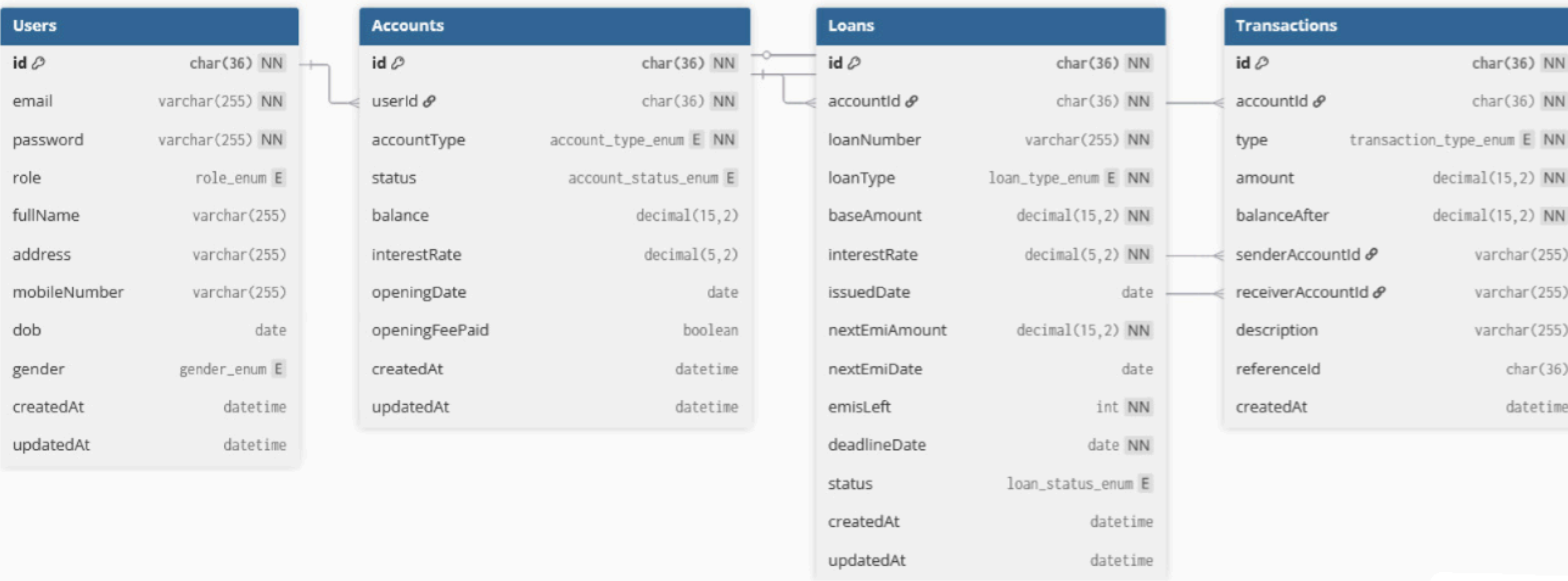


# ER Diagram





# Relational Schema Diagram



# Entity Relations in DB

- **User to Account (1 : N):** One User can have multiple Accounts (e.g., a Savings and a Current account). An Account belongs to exactly one User.
- **Account to Loan (1 : N):** One Account can be linked to multiple Loans. A Loan belongs to exactly one Account (where the funds are disbursed and EMIs are deducted).
- **Account to Transaction (1 : N):** One Account has many Transaction records (its history).

# Normalization & Functional Dependency

A Functional Dependency (FD) is a relationship between two attributes in a relation, where the value of one attribute (or set of attributes) uniquely determines the value of another attribute.

If attribute A determines attribute B, it is written as:  $A \rightarrow B$

Meaning: For any two tuples in a relation, if they have the same value for A, they must have the same value for B.

**1NF (First Normal Form):**

All attributes contain atomic (single) values and no repeating groups

**2NF (Second Normal Form):**

No partial dependency (non-key attributes depend on full primary key)

**3NF (Third Normal Form):**

non-key attributes depend only on the primary key and not on other non-key attributes (no transitive dependency).

**BCNF (Boyce–Codd Normal Form):**

A relation is in BCNF if every determinant is a candidate key, meaning only keys can determine other attributes.



# Normalization Steps Applied

- 1NF (First Normal Form):

All relations were checked to ensure that each attribute contains only single (atomic) values and no repeating groups or multivalued attributes exist.

All attributes such as names, balances, amounts, and dates store single values only, so all relations satisfy 1NF.

- 2NF (Second Normal Form):

Relations were examined to ensure that all non-key attributes depend on the entire primary key.

All other relations have single-attribute primary keys, so they automatically satisfy 2NF.

- 3NF (Third Normal Form):

All relations were checked to confirm that non-key attributes depend only on the primary key and not on other non-key attributes.

No transitive dependencies were found in any relation.

Therefore, all relations satisfy 3NF.

- BCNF (Boyce–Codd Normal Form):

All functional dependencies were analyzed to ensure that every determinant is a candidate key.

Since in every relation the primary key uniquely determines all other attributes, all relations satisfy BCNF.

All relations satisfy 1NF, 2NF, and 3NF, and since every determinant is a candidate key, all relations are in BCNF, ensuring minimal redundancy and improved data integrity.

# Functional Modules

## **Search Module**

Provides a unified directory for administrators to instantly lookup comprehensive user, account, and loan records.

## **Transaction Module**

Safely processes peer-to-peer transfers, fee deductions, and EMI payments using strict database transactions to guarantee data integrity.

## **Authentication Module**

Secures the platform with JWT-based sessions, bcrypt password hashing, and strict role-based access control.

# Tech Stack

## Frontend: ReactJs

React's component-based architecture allows for the rapid development of modular, reusable user interfaces.

## Backend: NodeJs

Node.js allows us to use JavaScript across the entire stack, creating a unified, full-stack development environment.

## DB: SQL

SQL is a tested RDBMS known for its strict ACID compliance and simplicity

# Conclusion

This project successfully creates a working digital bank. It makes it easy for users to open accounts, send money, and apply for loans, while giving administrators the tools they need to approve requests and monitor the system. By using a secure database and a modern web design, the system ensures that all transactions are safe, accurate, and easy to manage. Overall, this project shows how technology can be used to build a reliable and user-friendly banking platform.



**Thank You**